

HADOOP (HDFS)

07 June 2026 19:59

Big Data → Think of it as a problem that traditional DBs cannot handle efficiently.

↳ = Extremely large (huge in volume), grows exponentially with time and diverse that cannot be processed efficiently using traditional tools like Excel, SQL DB servers.

example - Imagine you're storing customer data for small shop where

Data Source	Daily size of Data
Customer records	10MB
Customer orders	50MB

A normal SQL DB can easily handle this. Now, imagine you're working at a company like Netflix or Amazon.

where Daily Size of the Data is humungous.

Data Source	Daily Data Size
User clicks	5 TB
Video watch logs	20 TB
Search history	10TB

i.e. ≈ 50 TB Data per day!!!

A single SQL server struggles to store and process this volume of data.

5 V's of Big Data — if any use case is satisfying these 5 V's then it's a big data use case & needs Big Data technology/system to handle the use case.

(1) Volume → How much data?
Size of the data is enormous could be in GB's, TB's, PB (1000MB - 1GB; 1000GB - 1TB, petabyte 1000TB - 1PB)

(2) Velocity → How fast data is generated?
Data is arriving continuously & very fast. It refers to the speed at which data is generated & processed.
- Depending on the business requirements: Data can be processed in Batch Mode (hours/days), near real time (mins/secs) or real time (milliseconds/sec - instantly).
- Batch mode - Data generated is collected for sometime (maybe hours/days) & then processed together.
- Near real time - Data generated should be processed after every 5-10 mins.
- Real time processing - Data must be processed almost instantly.
ex: UPI payment
send 1000 → Bank verifies → money transferred.

(3) Variety → What types of data?
It refers to different types and formats of data that an organization collects and processes.
- Structured Data: Data with fixed schema (rows & cols)
- Semi Structured Data: Data has some structure but not fixed rows or cols. Stored in JSON, XML files, NoSQL DBs.
- Unstructured Data: Data without a predefined format. - Images, videos, audio files, emails, PDFs etc.

(4) Veracity → Can I Trust this Data?
- It refers to the quality, accuracy, consistency & reliability of the data.
- It deals with issues such as missing values, duplicates, & inconsistent data.
- Data engineers improve veracity by data validation, cleansing, standardization and quality checks.

(5) Value → Is the data Useful to the Business?
- It refers to the business insights obtained from the data.
- example: A company stores 10TB of server logs every day!! That's a big data, but if nobody analyzes or uses it to solve a problem then
 { Data = huge
 Business Benefit = zero }
 So value is low.
- Why companies invest in Big Data? Not because they love storing the data, they want - Increased revenue, Reduce costs, improve customer experience, build models ⇒ These are Values.

examples of BigData

- 1) Social media - Data from posts, likes, shares etc
- 2) IoT - Real time data from devices like smart homes, wearables etc
- 3) Finance - Data from financial transactions, stock exchange trading systems

What is a cluster?

↳ imagine you've a CSV file containing 10TB of data

& your machine/system is of 16GB, 8Core CPU

configuration — So it cannot process 10TB of data all alone

- As - Data won't fit in the memory
- Processing would take hours or days
- Single machine becomes a bottleneck.

So instead of using 1 machine, we use multiple machines together to process this huge data. — & this group of machines is called as Cluster

machine/system → Nodes

ex. Suppose you've 1000 exam papers to check

Teacher A → Checks all 1000 papers
Time: 10 hours.

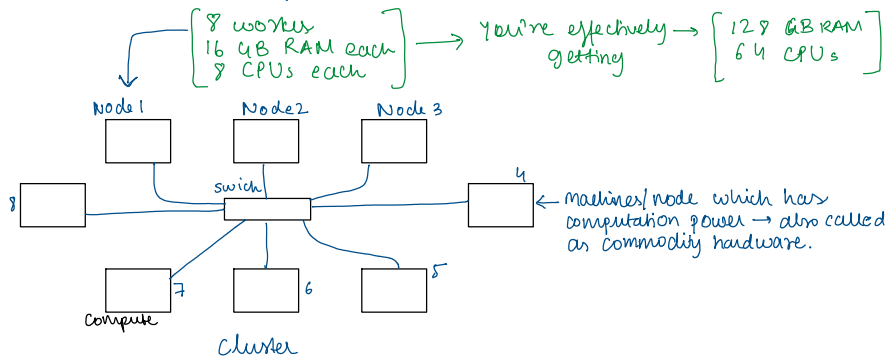
5 Teachers

A → 200 papers
B → papers
C → each
D →
E →

Time 2 hours → This is parallel processing

In distributed frameworks (like spark) → it will split my data into partitions & execute these across many machines in parallel.

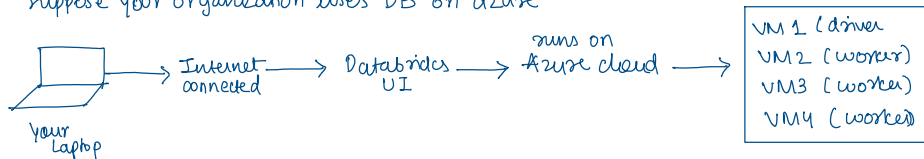
In Databricks → when you start a cluster



Note:-

These clusters are actually running on cloud server somewhere in the data center

suppose your organization uses DB on Azure



Those VMs (virtual machines) are actual computers/machine/commodity HW running in Azure data center

The data center may be physically located in India, Singapore, US etc depending on the cloud region you chose!

Now where is the data stored?

↳ Data is not stored in the cluster

Cluster is only used for computation → lives in Databricks workspace

Data → Storage account in Azure Data Lake Storage (ADLS)

In old systems →

Machine comes with storage + compute together

problem: If I need more computation or more storage → I need to buy bigger machine. which is expensive.

Modern Cloud →

storage } separate → To scale independently.
compute }

If I need more storage - Increase Data Lake size

If I need more compute - Add more workers/nodes/machine

{ You write your code on Notebook in DB, the code runs on the cluster on Azure. }
The cluster reads & write data from Azure data lake.

* let's understand storage a bit more -

Before cloud -

A company has a server room



After cloud -

Instead of buying servers yourself → You rent storage from

cloud providers like

- Microsoft Azure

- AWS

- Google cloud.

} They own huge data centers

A data center has → thousands of physical servers, inside those servers you have CPU, RAM, SSD/Hard disks etc.

So Data is physically sitting on SSD/Hard disk somewhere

The cluster is used for computation it temporarily reads the data process it using CPU & RAM and writes results back to storage. Cluster will disappear once the job is done, but the data remains stored permanently in storage.

So Data Lake → stores the data (ADLS)

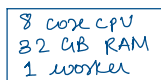
Cluster → computes the data. (Read, process & write).

Vertical & Horizontal scaling -

Vertical scaling → it increases the resources (CPU + RAM (memory)) of an existing machine

→ example:

Initially you had this machine

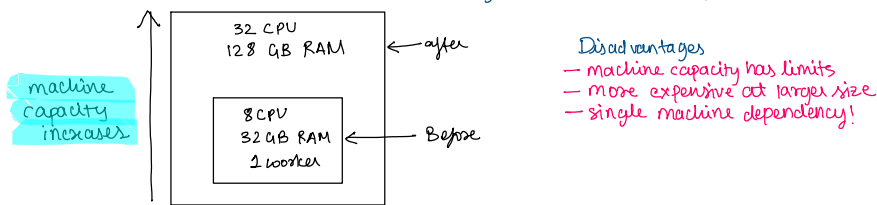


This was able to process 100 GB of data, but now its

Thus, we are in a predicament of having a machine, but not its struggling to process 10TB of data.

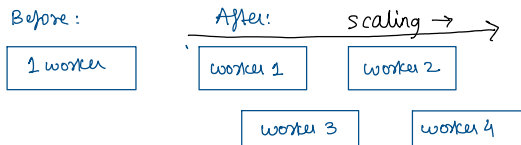
- We need to make this machine more powerful to process such huge data.

- Vertical scaling is one of the method → Makes existing system bigger/powerful



you upgraded the same machine (worker)

Horizontal scaling (Scale out) → It increases the no. of machines



- Instead of 1 strong worker, you can add more workers & data processing will be split across all the workers.

- Note: Spark was built around horizontal scaling.

Auto-scaling → The system automatically adds or removes machines based on workload.

example: Your Databricks cluster settings are

min workers = 2
max workers = 10

if the current workload → small, 2 workers will execute the job.

if a large ETL job starts → cluster will automatically grow to 10 workers to execute the job & when job finishes → cluster shrinks back to 2 workers.

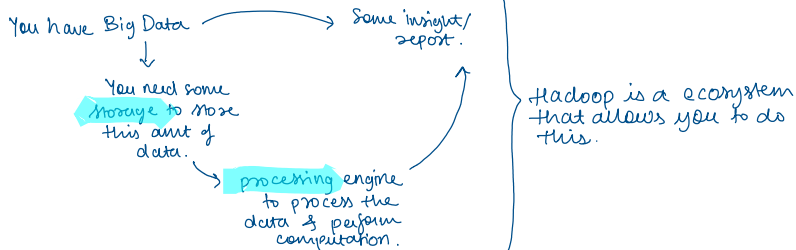
This is auto-scaling.

Why companies use auto-scaling?

without auto-scaling - 10 workers/machines running all day, even when nobody uses them

with auto-scaling → 2 workers normally, 10 only when needed so lower cloud bill.

HADOOP



→ Hadoop is a open source framework - collection of tools to handle big data (not a single software).

→ It allows storage & processing of huge amt of data across multiple machines.

Before Hadoop → Data was stored on 1 server
if server crashes → data lost
storage limited & slow processing

NOTE:
machines in Hadoop framework cluster used to have compute resources (CPU + RAM) along with storage.

Hadoop solved this by → distributed storage & processing across multiple machines introduced fault-tolerance & scalability.

→ Two main components of Hadoop (Big data framework) -

(1) HDFS → storage (This will take care of storing data across distributed machines/cluster)

(2) MapReduce → processing engine (it will take care of parallel processing of partitioned data across machines in a cluster)

(3) YARN (Yet another resource negotiator) → It's your resource manager, which will manage resources (CPU, RAM memory) of a cluster effectively.

example: suppose you have 4 machines in your cluster each of 8 Core CPU, 32 GB ram, then 3 Jobs arrive
How much CPU & RAM each Job needs → this will be decided by YARN. (resource allocation)

in Databricks, → it automatically manages cluster resources for you.

Properties of Hadoop

1) It stores & process the data in Batch (it does not support any framework or library that can handle streaming / real-time data)

Batch processing ex:- monthly spend statements from bank.

(collects all your debits & credits for a month & at month end send you the report)

- Electricity bill that we get, it stores the data of per day consumption & computes the tariff and send you the bill at every month.

2) Scalability → can store and distribute large data sets across many machines.

⇒ Cost effectiveness → designed to run on inexpensive, commodity hw

4) fault tolerance, open source.

HDFS

↳ It follows a master slave architecture, main components are -

(1) NameNode (the master)

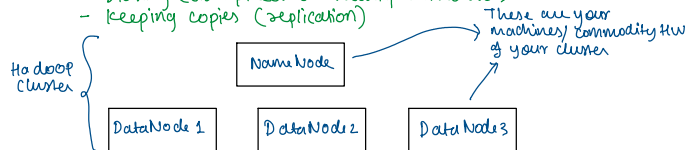
(2) DataNode (the slave)

↳ What problem HDFS is solving?

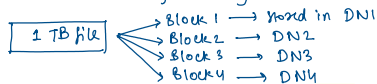
Suppose you have 1TB of customer data, storing it in 1 machine is risky as - Data lost if machine crashes.

HDFS solves this by -

- Splitting the files into pieces (blocks)
- Storing each piece on multiple machines
- Keeping copies (replication)



- DataNode → Storage Machines, these are the worker nodes. They actually store the data blocks.



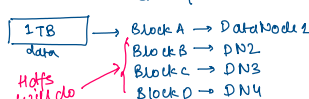
remember → DN contains the actual data/replicas of data.

- NameNode → This node is like the Brain of HDFS

→ It does not store the actual file contents instead stores metadata

means information about data

→ example :-



It is not going to participate in the data storage or processing.
The NameNode knows -
filenames, folder structure, which block belongs to which file & which datanode has each block.
- To maintain the catalog/metadata of files.

How does a hadoop cluster know which machine will act as Namenode & which one's will be datanodes?

↳ Hadoop doesn't automatically decide, we need to configure while installation that which machine will be your Namenode & which datanode

A machine is just a server

& Hadoop runs software processes on different servers

ex: machine A → namenode process
machine B → Datanode process
machine C → Datanode process

So, according to the configuration it will decide which server will run a namenode process & which servers will run Datanode process

Then you start Hadoop services

once the services starts → Datanode connects to the namenode & registers it
" Hello,
I am Datanode 3, I have 10TB of storage"

→ Namenode starts listening for datanode registrations.
" I am the manager, workers reports to me"

Blocks

↳ Smallest unit to store.



Who will decide in how many blocks the data should be divided?

↳ configuration

Block size config.

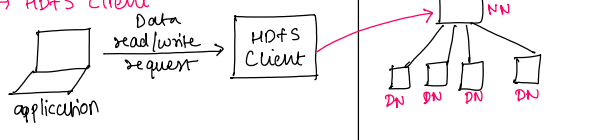
default configured block size is 128 MB in hadoop 2.0.

→ It's not like all the datanodes will store the blocks in even manner (like 8 blocks, 4 nodes → so 2-2 each X).
it depends on the storage capacity of the DN.

* Hadoop HDFS is built to prevent data loss if any DN server/machine crashes - Fault Tolerance

Who is responsible to ~~get~~^{receive} the operations from application/user (read/write 10GB of file from HDFS)?

↳ HDFS Client



→ it's a software/library that lets application to interact with the HDFS / hadoop cluster.

→ it is responsible for splitting the received data into blocks logically according to configured block size → Then give it to namenode which decides which DN should store each block.

each DataNode servers/machines sends heartbeats signals/health report to the namenode every few seconds, so Namenode knows which system/server is alive up & running & which server (DN) is dead/crashed!

If it doesn't receive any signal from DN, then NN doesn't consider it to store the block until an unless it is up & running.

Interview Q's -

You have a file of 612MB, and block config is 128MB - so you split the file into 5 blocks

Block 1 - 128MB
Block 2 - 128MB
Block 3 - 128MB
Block 4 - 128MB
Block 5 - 100MB

→ can the remaining 28MB in block 5 used by another file partition?

Ans - NO

Common misunderstanding → HDFS allocates exactly 128MB on disk for every block X

Block size = 128MB (is the maximum size) i.e. it means "keep adding data to current block until it reaches 128MB, then create a new block"

And → HDFS doesn't allow multiple files in one block. as each block is a unit of replication, metadata for a file.

i.e. NameNode stores metadata like-

File A
- Block1
- Block2
- Block3
- Block4

if block contained pieces of multiple files

Blocks →
- file A (100MB) X
- file B (28MB)

then metadata becomes more complex, replication & recovery becomes harder & even parallel processing becomes hard.

So, A Block belongs to only file

Key Terminologies & components of HDFS

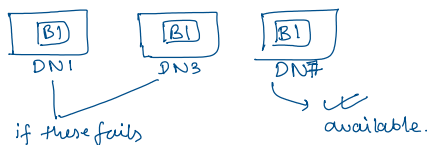
(1) Replication management -

→ To prevent the data loss if any commodity HW fails (achieve fault tolerance) HDFS creates multiple copies of each block.

File A
└ Block1 (DN1) → This node crashes, so Block1 is lost & so is part of your file
└ Block2 (DN2)

- By default → HDFS uses replication factor of 3. (including original copy)

for ex: Block 1 will have 3 copies stored in 3 different datanodes



- NameNode manages the replication, as it stores the metadata as-

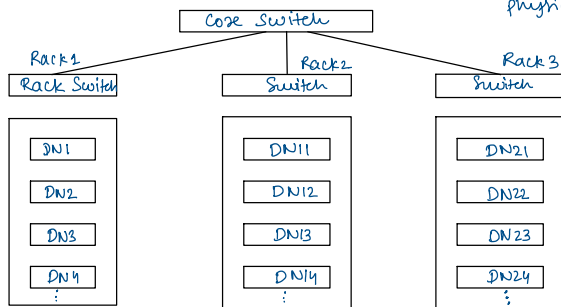
B1 → DN1, DN3, DN7.
B2 → DN2, DN5, DN6.

it knows which DN are alive, which block exists & where replica of each block is stored.

- So, namenode continuously monitors datanodes through heartbeats and automatically creates or removes replicas of block so that no. of replicas of each block maintains the configured replication factor (3) → ensuring fault tolerance & high availability of data.

- 1GB of file → in HDFS it actually stores 3GB of data considering Replication of 3.

(2) Rack awareness → In a Data center, Hadoop cluster with 1000 servers physically are arranged in racks,



All server/machines/Datanodes → inside a RACK shares one switch.

Communication inside a rack →

Suppose DN1 wants to send data to DN4

DN1 → Rack 1 switch → DN4
(This is fast, cheap & low latency.)

communication across racks →

DN1 wants to send data to DN22

DN1 → Rack1 switch → Core switch → Rack3 switch → DN22

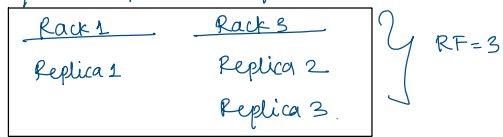
This is slower & consumes expensive backbone bandwidth.

Cross-rack traffic is something datacenter try to minimize.

→ If all the replicas are stored in DN of same rack & if rack switch dies → All machines of the rack becomes unreachable even though the machine is fine but rack is isolated.
→ Data is lost

→ Rack awareness is a mechanism in HDFS that intelligently places block replicas across different racks (to protect against rack failures)

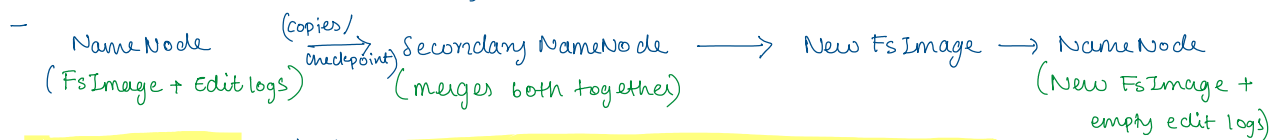
- For $RF=3$, HDFS places 1 replica on one rack and 2 replicas on another rack (To minimize the cross-rack network traffic) & improving read/write operation performance.



- HDFS deliberately trades a little fault tolerance (not using 3 racks to store the replicas, uses only 2) to avoid network congestion & (better n/w efficiency).

(3) Secondary NameNode →

- Biggest disadvantage of master-slave architecture is → master's single point of failure
- If Namenode dies → you'll loose access to your data stored in different DN's (as it stores the metadata)
- How to prevent this? → Secondary namenode



- Secondary NN → periodically merges the FsImage & edit logs to create checkpoints, preventing edit logs from becoming too large and reducing Namenode recovery time

(Note - It is not a backup node, & does not provide backup if namenode fails).

FsImage → file system image

→ It contains metadata (snapshot of the entire HDFS namespace at a particular point in time)

→ like - $\left\{ \begin{array}{l} /data/sales.csv \\ /logs/app.log \\ /data/customers.csv \\ \text{Sales} \rightarrow \text{Block 1, Block 2, Block 3} \\ \text{app.log} \rightarrow \text{Block 4} \end{array} \right\}$ } snapshot/ FsImage

Edit logs → $\left\{ \begin{array}{l} \text{Create file A} \\ \text{Delete file B} \\ \text{Rename file A to file Z} \\ \vdots \\ \text{etc.} \end{array} \right\}$ } all the updates / edits made are stored as logs.

- Example:-
problem:- Over time, Namenode will have

$\left\{ \begin{array}{l} \text{FsImage} \\ + \\ \text{millions of edit log entries} \end{array} \right\}$

- & if Namenode crashes, on restart it must load FsImage & replay millions of edit. & recovery of system becomes slow
- To reduce NN recovery time → Secondary Namenode periodically create a checkpoint of NN's FsImage & its edit logs.
- first it copies the metadata (FsImage with Edit logs) from namenode

- merges them i.e

if FsImage: $\left\{ \begin{array}{l} \text{File A} \\ \text{File B} \end{array} \right\}$
edit logs: $\left\{ \begin{array}{l} \text{create File C} \\ \text{delete File B} \end{array} \right\}$ } combine them → New FsImage $\left\{ \begin{array}{l} \text{File A} \\ \text{File C} \end{array} \right\}$
Edit log: empty

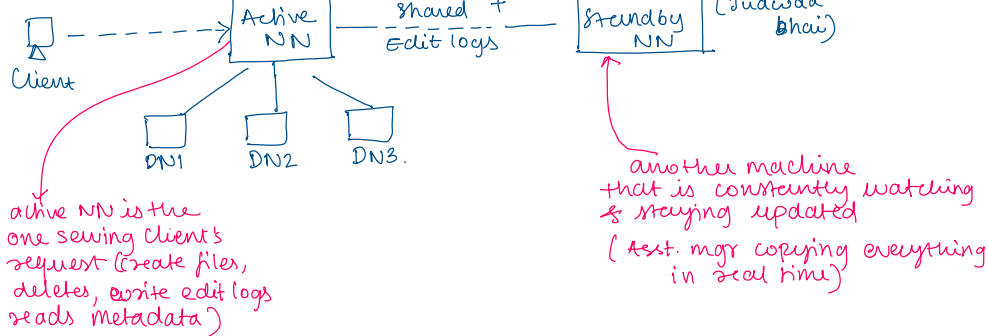
- Secondary NN sends the checkpoint (i.e New FsImage) back to the namenode & NN replaces old FsImage + logs with New FsImage + empty edit logs.

- Now restart of NN will be faster !!

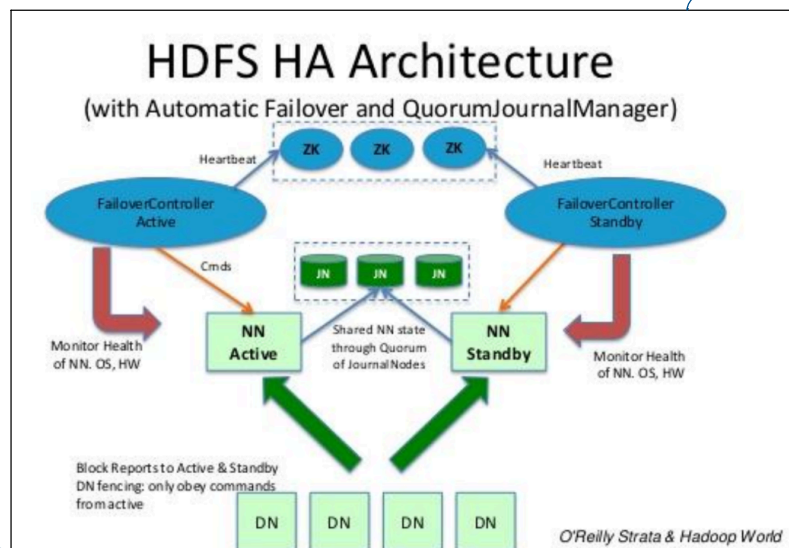
In short, Secondary namenode helps in checkpointing of NN's FsImage + edit logs & reduce recovery time of NN if it crashes by providing the New FsImage periodically by checkpointing, it doesn't take over NN's job if it crashes !!

(4) Standby NameNode →

- A Standby namenode is a synchronised backup namenode used in Hadoop High availability (HA) architecture (introduced with Hadoop 2.0) before this → secondary NN.
- It continuously reads merged edit logs and maintains up to date copy of Active Namenode fsImage.
- If the Active NN fails, the Standby namenode automatically takes over eliminating single point of failure.
- It also performs checkpointing, replacing the need of secondary namenode.



- Working Manager
- In secondary NN, if Active NN fails, there will be a downtime of (~ 2hr) to set up a new system that will act as NN & copy the fs image from the secondary namenode.
- But with standby NN, if Active NN fails, there is no downtime as standby NN automatically takes over!! as new NN (active).



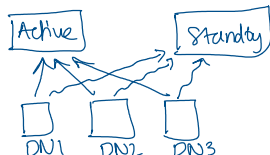
main goal of this HA architecture was to handle SPOF (single point of failure of NameNode/master)

There are 3 layers

- Top - Zookeeper cluster (ZK1, ZK2, ZK3) & failover controllers.
- Mid - Active NN & standby NN (with Journal nodes)
- Bottom - Data nodes.

Working

- 1) Suppose Client wants to create sales.csv
Client → Active NN
(Only Active NN serves the client)
- 2) Then Active NN updates the metadata i.e. sales.csv → Block A
But instead of storing this information locally it writes the edits into the Journal nodes (JN1, JN2, JN3)
(shared space for metadata changes/edit logs)
- 3) Active NN waits until a majority of Journal nodes acknowledges (this majority is called Quorum)
- 4) Standby NN reads the edits / metadata from the JN & applies the same edits, Thus both the NN now knows:
sales.csv → Block A → DN1
 → Block B → DN2
- 5) Datanodes send the block report to both the namenodes



Standby node is completely up to date.

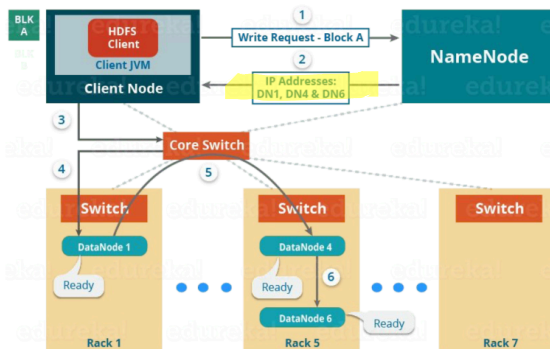
- 6) Each Namenode has ZKFC (zookeeper failure controller) that continuously monitors Namenode - Process, CPU, OS, memory & they keep sending heartbeats to zookeeper
- 7) There are 3 zookeeper server → their job is to make sure only 1 NN is active; They know which NN is active & which one is standby.
- 8) Now if NN Active fails :- The active ZKFC stops sending heartbeat to zookeeper servers - & zookeeper notices that heartbeat is missing & concludes Active NN (NN1) has died & elects standby NN (NN2) to

become Active NN

- 5) **Failover is very fast** → As Standby already has the metadata which was continuously reading & updating itself from the journal nodes & received blocks reports

Write operation in HDFS - Step 1

Setting up HDFS - Write Pipeline



- Assume you want to store Block A (RF=3)

1) Client sends write request of Block A to NN. (note client is only sending the request not the data)

2) NN decides where to store the block & its replicas & on which Rack & DN's. Using → Rack awareness algo, available space & healthy node.

& returns IP addresses of the location where 3 replicas of Block A has to be stored. NN Job ends here, it is not participate in transferring the data.

Client then establishes a TCP connection to DN1 through the N/w

Client → Core switch → Rack switch → DN1
DN1 replies ready → to receive Block A.

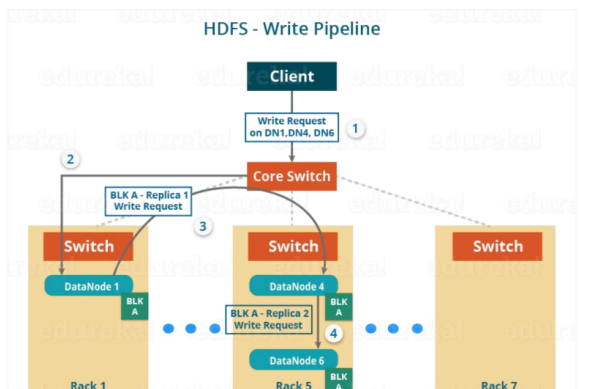
6) then for RF of 3, DN1 builds connection to DN4, DN4 replies ready ⇒ Client → DN1 → DN2

7) third replica is needed → DN4 opens connection to DN6.

Now complete pipeline is established: Client → DN1 → DN4 → DN6

Write operation in HDFS - Step 2

HDFS - Write Pipeline



→ Now the pipeline is already established, client just sending BLK A to DN1

(Note client sends data only to DN1, it does not send separately to DN4 & DN6)

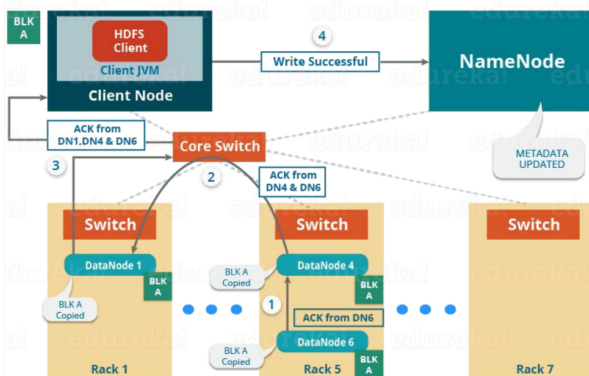
> DN1 receives BLK A from the Client & writes it locally (Replica 1) & forwards the same to the DN4

> DN4 receives Replica 2 write request from DN1 - it stores & forwards it to DN6.

Then DN6 receives Replica 3 write request from DN4 - the DN6 simply stores the replica locally.

Write operation in HDFS - Step 3

Acknowledgement in HDFS - Write



This is the ack phase of HDFS write pipeline.

Current state: DN1, DN4 & DN6 has the Block A

DN6 sends ack to DN4 (I've stored my copy)

DN4 sends ack to DN1 → Both me & DN6 have stored the block

DN1 sends ack to Client.

Client after receiving the final ACK from DN1, it informs namenode & namenode updates the metadata. (edit logs)

Read Operation in HDFS

What is happening?

Suppose a file has two blocks:

- Block A
- Block B

1 Client asks NameNode "I want Block A and Block B".
2 NameNode looks in metadata and tells the client where those blocks are available.
(Example: Block A → DN1, Block B → DN3)

3A Client chooses a DataNode that has Block B (DN3 in this case).

3B Client chooses a DataNode that has Block A (DN1 in this case). (Important: Reads happen in parallel from multiple DataNodes.)

4 DataNodes send the blocks directly to the client.

Key Points

- Client talks to NameNode only at the beginning.
- Data flows directly between DataNodes and the client.
- Reads happen in parallel from the closest/faster DataNodes.
- NameNode is NOT in the data path.

HDFS - Read Architecture

